Harnessing Neural-Ordinary Differential Equations for Pharmacometrics: Dynamical Modelling in the Age of AI

*James Lu and Isaac Yeoh. Bioinformatics Institute, A*STAR, Singapore. Email: james_lu@a-star.edu.sg*

Abstract

Differential equations have long been used as the language of pharmacometrics (PMx). However, the rise of high-dimensional, multi-omics data and the increasing complexity of biological knowledge demand more flexible and scalable modelling approaches. This tutorial introduces Neural-Ordinary Differential Equations (Neural-ODEs) as a transformative methodology that bridges classical dynamical modelling with deep learning. By replacing algebraic equations with highly expressive, differentiable neural networks, Neural-ODEs provide a versatile canvas capable of learning complex pharmacological dynamics.

We explore the mathematical foundations of Neural-ODEs, including their encoder-decoder architectures and data augmentation strategies tailored for longitudinal data. Additionally, we discuss the integration of mechanistic priors through Pharmacology-Informed Neural Networks (PINNs), which embed causal structures and biological knowledge to enhance model predictivity and sample efficiency. The paper highlights frontier opportunities for this methodology in advancing drug development. Finally, we propose how Neural-ODEs together with AI agents can empower pharmacometricians in the next generation of Model-Informed Drug Development.

1. Introduction

1.1 The Role of Differential Equations in Pharmacometrics

Differential equations (DEs) are the *lingua franca* of pharmacometrics (PMx): as a core part of its foundation, the discipline is concerned with kinetic concepts such as turnover, half-life and progression, which are only precisely conveyed in a mathematical language describing the rates of change of various pharmacological and biological processes. Through the DEs, such models can be reasoned about analytically and used in simulations of "what-if" scenarios, critical to applications of Model-Informed Drug Development (MIDD) (Lu and Desikan, Quantitative Systems Pharmacology Modeling Amid the Rise of Agentic AI 2026).

DEs have become so firmly established in PMx that the terminology "structural model" is used to refer to the Ordinary Differential Equation (ODE) system being used. Furthermore, the commonly used parameters in such ODE systems (e.g., CL, V, ka in pharmacokinetic (PK)

models) have been mapped to physiological interpretations, providing a shared collective vocabulary for communication within the discipline, thereby enabling PMx to mature beyond a collection of data fitting algorithms and results - into a vibrant scientific community.

1.2 Motivation: Why Now and Why Neural-ODEs

Despite the growth in impacts and the size of scientific community of PMx over the years, there is now an ever-increasing expectation of how PMx can bring value to the rapidly changing paradigm of drug discovery & development – especially in this era of Artificial Intelligence (AI) (Shahin, et al. 2025) (Terranova, et al. 2024). In particular, relentless exponential technological forces— (1) surging computational power; (2) plummeting sequencing costs; (3) exploding scientific knowledge base—are fundamentally shifting the bedrock of the drug development landscape (see Fig. 1).

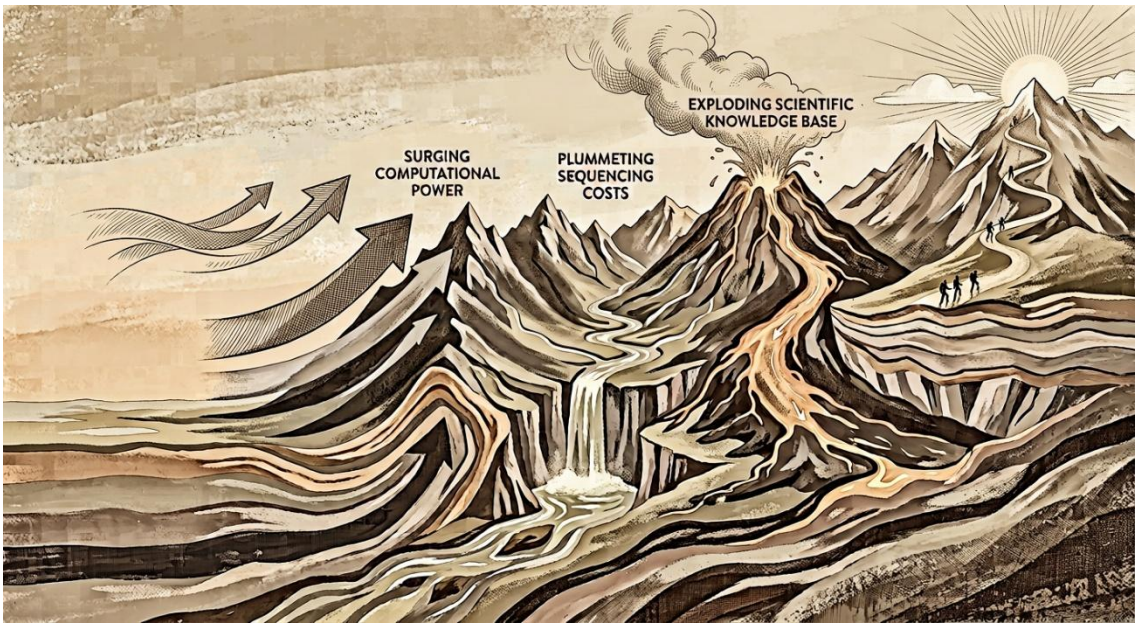


Figure 1. The exponential technological forces jointly reshaping the landscape of drug discovery & development.

Consequently, pharmacometricians are facing mounting pressures to:

1) **Accelerate delivery timelines**: the need to dramatically compress modelling and decision-making cycles to keep pace with automated data modelling enabled by broader adoption agentic AI within computational disciplines;
2) **Harness high-dimensional data**: the influx of rich, complex datasets requires pharmacometricians to move beyond standard models and extract deep, actionable insights from multi-omics data streams.

3) **Integrate expanding biological complexity**: become cross-disciplinary integrators to weave ever-richer biological knowledge into pharmacometrics models.

A methodological development that combines the elements of PMx being rooted in ODEs, while addressing the growing demands on PMx, is Neural-ODEs (Chen, et al. 2018). As compared to many other powerful deep learning (DL) model architectures for sequential and time-series data (such as Transformers), it has the advantages of adopting a dynamical systems centric view, hence offering advantages in handling temporally irregularly sampled data as well as providing a flexible yet mechanistically constrained modelling canvas. There have been a number of different applications in pharmacometrics, ranging from pharmacokinetics (PK) (Lu, Deng, et al. 2021) to pharmacokinetics/pharmacodynamics (PK/PD) (Lu, Bender, et al. 2021), disease progression (Laurie and Lu 2023) and digital health applications (Pakravan, et al. 2024). While there is a whole garden of related variants of methodologies that combine concepts from differential equations and DL, due to the scope of this tutorial we will not provide an overview of the similarities and differences. However, many of the concepts introduced here will also apply in these other contexts, upon suitable modifications.

2. Foundations of Neural Ordinary Differential Equations

2.1 Representation Space

In PMx modelling, each model is represented as a given ODE system:

$$\frac{dy(t)}{dt} = f(y(t), p), \qquad y(t) \in \mathbb{R}^n, p \in \mathbb{R}^m, \quad (1)$$

where $f(\cdot,\cdot)$ is the vector field. With distinct sets of parameter values $p \in \mathbb{R}^m$, the model can exhibit a range of different dynamics. For instance, a model with linear kinetics can exponentially decay or grow with time, depending on the parameter values. More sophisticated PK/PD models can exhibit complex dynamics, ranging from hysteresis to oscillations as the parameter values span from low to high. However, provided the vector field $f(\cdot,\cdot)$ of the ODE system is fixed, there is a limited range of dynamics it can exhibit.

In contrast, in Neural-ODEs (NODEs), the system can be expressed as:

$$\frac{dy(t)}{dt} = f_\theta(y(t), p), \qquad y(t) \in \mathbb{R}^n, p \in \mathbb{R}^m. \quad (2)$$

Here, $f_\theta(\cdot,\cdot)$ is a vector field represented with a neural network (NN), parametrized by *trainable weights,* θ. See Fig. 2 for an illustration, depicted with a simple fully connect network.

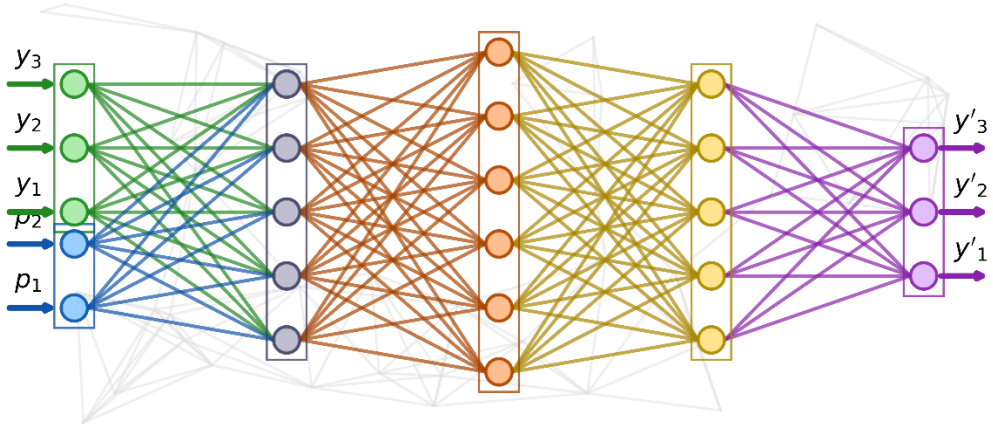


Figure 2. An illustration of how the vector field f_θ of a Neural-ODE can be represented with a neural network, with the trainable weights θ represented by the lines between the nodes. Note the dimension of the output layer (y') matches that of the first partition of the input, y.

Why might one want to replace the algebraic expressions typically used for ODEs, and why might NNs be useful in this context? An answer to the first part of the question is that while algebraic expressions often express certain mechanistic knowledge and/or assumptions of the system being modelled, they often constrain the kind of dynamics that can be represented and may, in fact, induce bias (Turner, et al. 2026). Furthermore, the iterative process of model initialization, diagnostics, selection and refinement can be a highly time- and resource-intensive one, with the challenge that gaps in data-fit will often remain, with the modelling process sometimes coming to an end with no clear understanding whether the initial set of assumptions are correct or not.

A reason why deep NNs can be a good choice is that they are highly *expressive* in the sense of being able to span a wide space of dynamics with a given architecture. NNs are also particularly well-known for their ability to create hierarchical representations of the data; a good example being image recognition NNs, which has shown to learn to combine low level features such as edges and curves into higher level, composite features such as the outline of a face. Moreover, effective training algorithms have been developed so as to learn the weights θ in a differentiable manner. Such capabilities could be potentially leveraged to recapitulate multiscale dynamics that are often present in biology, disease processes and/or pharmacological effects. Fig. 3 gives an illustration of how each PMx model inhabits an "island" of model dynamics, versus an encompassing range of dynamical behaviours that are possible with neural-ODEs, making it possible to traverse between these islands of dynamical spaces inhabited by the individual PMx models, as well as beyond them.

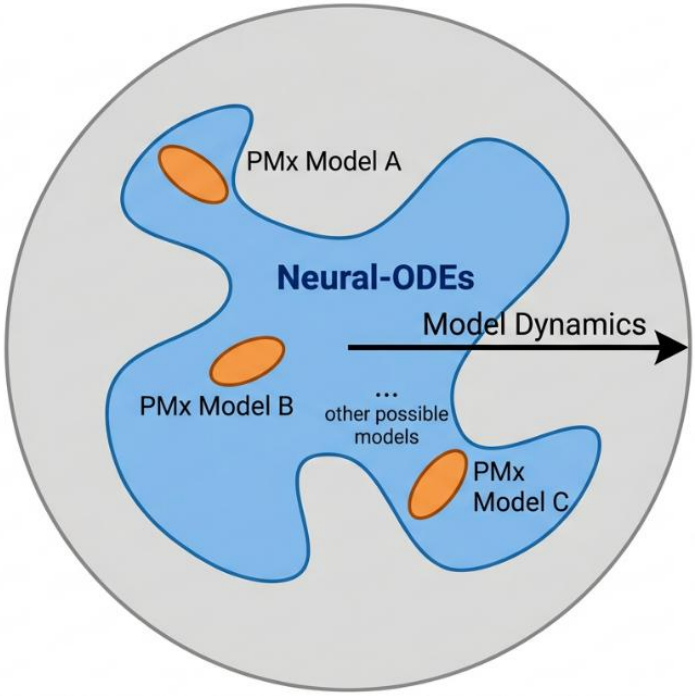


Figure 3. An illustration of the space of model dynamics reproducible with individual pharmacometrics (PMx) models, and how the space of model dynamics of neural-ODEs can encapsulate them and beyond. The differentiable formulation of neural-ODEs and the training algorithms enable a continuous path between spaces inhabited by distinct PMx models.

2.2 Notes on Mathematical Formulation

In the standard dynamical models that arise from PMx/QSP/PBPK, as represented in eqn. (1), the parameter vector p needs to be determined: there are various techniques, including population modelling of longitudinal data and other methods whereby the literature data is used to inform either components of p or their prior distributions. Furthermore, the initial condition $y(0) = y_0$ also needs to be determined. If the whole initial state y_0 is measured, then this is straightforward. However, if components of y_0 correspond to latent, unmeasured quantities, then several approaches are possible: either y_0 could be derived so that equilibrium condition is assured (i.e., $f(y_0, p) = 0$), or they are partially estimated from existing data.

One conceptual advantage of leveraging Neural-ODEs in PMx is that the notions of parameter vector p and initial condition $y(0) = y_0$ are directly carried over. However, rather than determining p via the above-mentioned methods, they are determined by passing the longitudinal data into a NN (dynamics) encoder, $p = DynEnc_\theta\big([(t_0, \psi_0),\ (t_1, \psi_1), \dots, (t_q, \psi_q)]\big)$, where the tuples (t_i, ψ_i) are *(time, measurement data)* pairs – with other possible variations depending on what other prior information available. In particular, $DynEnc_\theta$ are typically encoders that can work with time-series data of arbitrary length, such as Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) and many other possible architectures. Additionally, rather than setting the initial state to a fixed value, it is also estimated from data reflecting the initial state

being modelled, e.g., $y_0 = ICEnc_\theta([(t_{-1}, \psi_{-1}), (t_0, \psi_0)])$ whereby the *(time, measurement data)* tuples that are prior to, and at the time of treatment, are included and fed to the initial condition encoder. For instance, tumor size data prior to the time of treatment is given – reflecting the initial state of the patient. If there were tumor imaging and other high-content data available, they can also be fed into $ICEnc_\theta$ if an appropriate NN is used. This illustrates how Neural-ODE can generalize existing algebra-based ODE models, by "learning" abstractions of the initial (high-dimensional) state that can then be represented in a low dimensional y_0. Furthermore, much of the existing intuition of the modellers can be leveraged: e.g., what dimensionalities for parameters and state space are needed to describe the available data, and most suitable for the task at hand.

2.3 Encoder-Decoder Architecture

Putting the pieces together, we have in essence an *encoder-decoder* model architecture (and variants thereof), whereby the encoders "abstract" or creates a low-dimensional representations ($y_0 \in \mathbb{R}^n, p \in \mathbb{R}^m$) from longitudinal data and initial state data, each of which can be of arbitrary number and/or high dimensional; subsequently, the observed complex and/or high content dynamics is reproduced as well as possible. Please see the Fig. 4 for a schematic of the overall architecture.

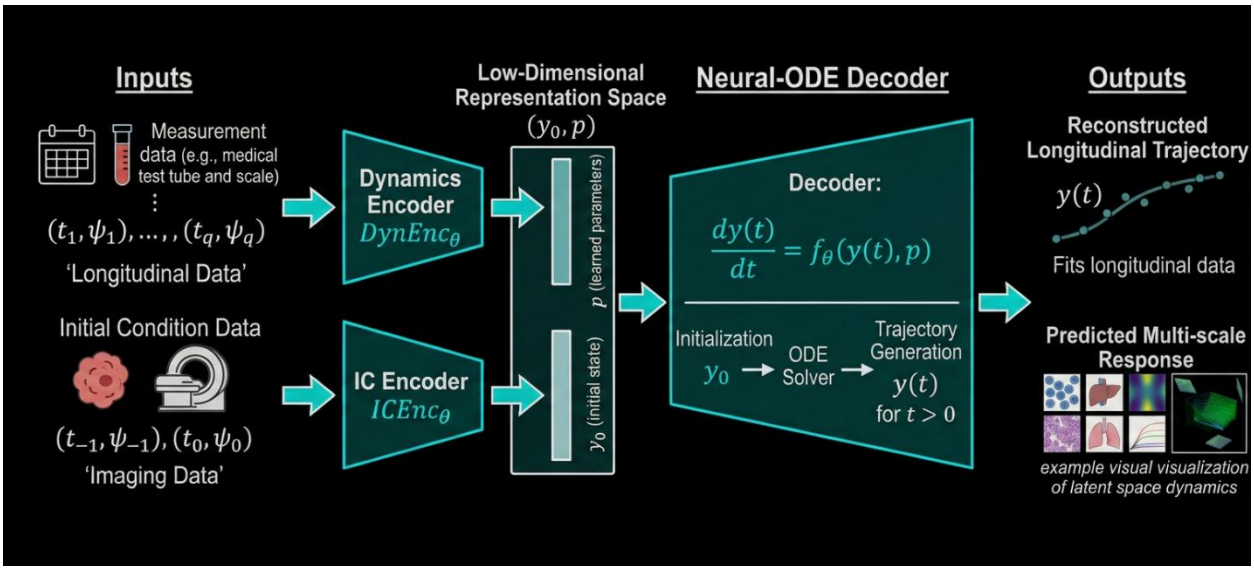


Figure 4. The encoder-decoder architecture of neural-ODE, showing how data is fed into encoders to derive low-dimensional representations for the kinetic parameter vector and initial conditions. Using those vectors, the neural-ODE model can reproduce the observed dynamics.

Here, we give an illustration on the use of a Neural-ODE: in particular, we consider an instantiation of Fig. 3 whereby we aim to show that the dynamics from a Simeoni tumour dynamics model (Simeoni, et al. 2004) can be encapsulated using a neural-ODE consisting of a vector field represented by a shallow NN consisting of 3 linear layers:

$f_\theta(\cdot,\cdot)$: $Linear([\mathbb{R}^1||\mathbb{R}^8] \to 32) \to SELU \to Linear(32 \to 32) \to SELU \to Linear(32 \to 1)$.

By training on simulated tumour dynamics, we see that the weights in the neural-ODE model are tuned to reproduce the Simeoni model, as demonstrated in Fig. 5 with a few sample reproductions.

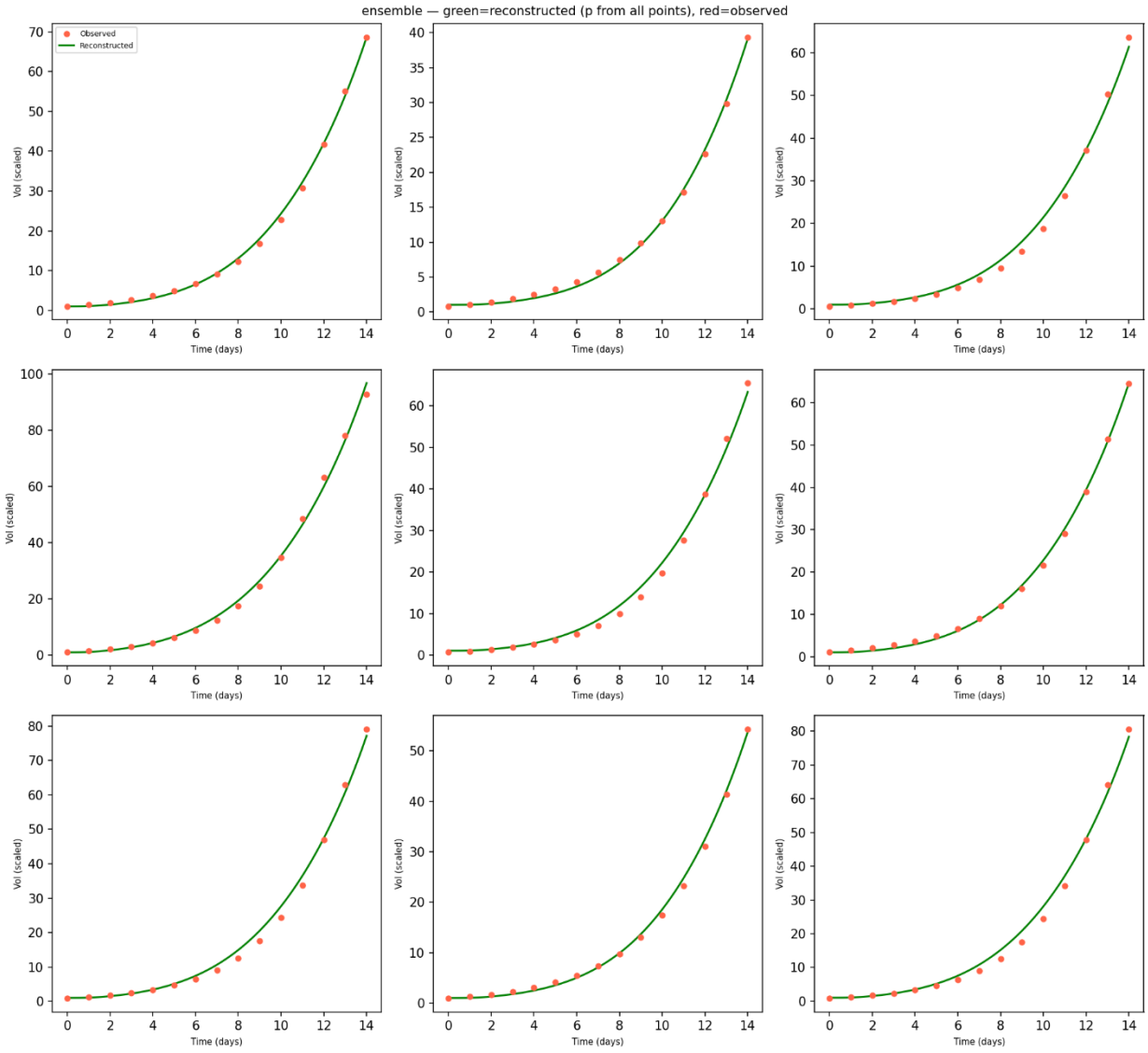


Figure 5. By feeding the Simeoni model simulations (shown as red dots), we show that a neural-ODE model with a simple architecture consisting of 3-linear layers was able to generate dynamical trajectories (solid green lines) reproduce samples of trajectories generated by the Simeoni model.

2.4 Enriching Training Data via Data Augmentation

A key aspect in the training of ML models is *data augmentation*: the process whereby the training data is enriched in size and diversity by creating modified copies of existing data. As DL models may have many trainable parameters (or weights), there is a tendency to memorize

features of the data rather than learn the general pattern. Data augmentation can address this tendency by generating valid variations of the training data, and in doing so, steer the model in directions that will help its generalization capability.

For time-series data, which take the form of $[(t_0, \psi_0),\ (t_1, \psi_1), \dots, (t_q, \psi_q)]$, any subset of the data that includes the first data point (t_0, ψ_0) as well as at least one subsequent data point (i.e., $(t_i, \psi_i), i \geq 1$) would also constitute a valid time-series data. Let us consider the following simple data augmentation scheme (Laurie and Lu 2023), that would encourage the model to make the best *next-data-point* prediction:

Input data window	**Prediction window**
$[(t_0, \psi_0),\ (t_1, \psi_1)]$	$[(t_2, \psi_2), \dots, (t_q, \psi_q)]$
$[(t_0, \psi_0),\ (t_1, \psi_1), (t_2, \psi_2)]$	$[(t_3, \psi_3), \dots, (t_q, \psi_q)]$
…	…
$[(t_0, \psi_0),\ (t_1, \psi_1), \dots, (t_{q-1}, \psi_{q-1})]$	$[(t_q, \psi_q)]$

Therefore, under this data-augmentation scheme, a time-series with q number of data points will result in an augmentation factor of $(q - 1)$.

If one were to consider the general data augmentation scheme whereby the first data point as well at least one other subsequent data point is included, then there are $2^{(q-1)} - 1$ ways of doing so – which makes the augmentation factor that increases exponentially with the number of data points. For instance, for q=5, the augmentation factor is 15; for q=6, the augmentation factor is 31.

2.5 Incorporating Mechanistic Knowledge

While NODEs have flexibility to learn directly from the observed longitudinal data, for many pharmacological applications there are mechanistic and/or biological knowledge in the form of *priors* that are important to incorporate. The use of such priors brings about three key potential benefits: (1) Improving the ability of the model to make predictions in the setting of limited data: that is, *Predicivity*(*Data-Driven* + *Mechanistic Prior*) > *Predictivity*(*Data-Driven*) (Bazgir, et al. 2023); (2) Enable better generalizability. For instance, from seen to unseen dosing regimens (Lu, Deng, et al. 2021) and across different biological contexts (such as *in-vitro* and *in-vivo*) as needed for translational applications; (3) Make the model predictions actionable by bringing causality into view.

There are two broad approaches for incorporating domain knowledge into NODEs:

1. Higher level architectural design:

   The concept of Pharmacology-Informed Neural Network (PINN) (Laurie and Lu 2023) proposes that high level aspects of network architectures and/or data augmentation should be determined by causality and other key modelling considerations. E.g., if we are to encapsulate the causal diagram of $dose \rightarrow PK \rightarrow PD$, how should the dosing change the network state and what data should be fed into the PK and PD encoders? Furthermore, how should the NNs for PK and PD vector fields be connected to be consistent with the assumption that former drives the latter (Lu, Bender, et al. 2021)? As an example of a key modelling assumption that need to be accounted for: suppose the outputs of the encoder should acquire a physical meaning of kinetic parameters, then they need to have the physical units of inverse time – which can be "taught" to NNs via performing temporal data transformations and their associated augmentations (see (Laurie and Lu 2023)). Other considerations, say, of requiring certain components of the encoder outputs being dimensionless parameters could also be "taught" to NNs.

2. As mechanistic scaffolds of the NNs:

   A standard choice of the NN architecture for the vector field is the *multilayer perceptron* (MLP), consisting of a set of linear layers whereby each node is connected to every other node. However, for many biological and pharmacological modelling applications such NN architectures may not be ideal as they do not leverage mechanistic priors of the known relationships between the variables being modelled. As an example of biological knowledge, if the variables are gene or protein expressions, the associated gene-gene-interaction or protein-protein-interaction networks can help to inform what subsets of interactions are supported by existing data – such information can be used to either constrain the topology of the NN or help guide the direction of the high dimensional search space. NN methods that can incorporate directed graphs (Rossi, et al. 2024) could be used as mechanistic scaffolds within NODEs. While there has been work done in this direction for biological modelling (Hossain, et al. 2024), significant opportunities await in QSP applications.

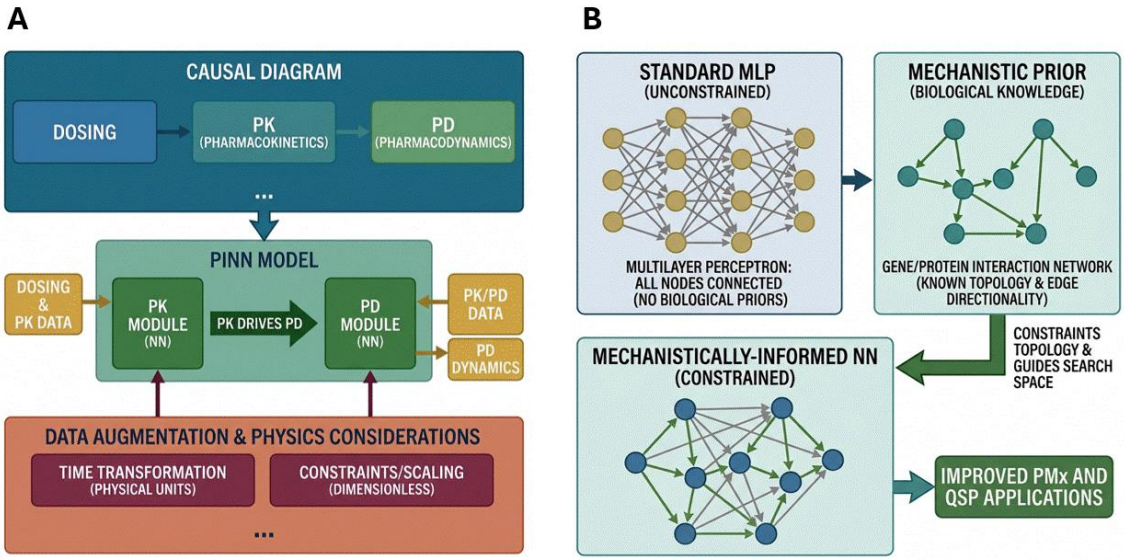


Figure 6. Schematic illustrating how mechanistic knowledge can be incorporated into neural-ODE models. Panel A: Pharmacology-Informed Neural Networks (PINNs) incorporate key pharmacological concepts in the form of causal diagrams as well as physical meaning of the variables being represented into the high-level architectural design and design process. Panel B: detailed mechanistic knowledge of the system being modelled can be used as a prior, to serve as a scaffold for the underlying NN and hence enhance its application in PMx and QSP applications.

4. Opportunities for Growing the Role of Neural-ODEs in Drug Discovery & Development

4.1 Increasing Modelling Throughput and Sample-Efficiency

The adoption of a new modelling approach like Neural-ODE offers the opportunity to re-examine how and where PMx modelling has traditionally been applied in the drug development setting. Typically, early on during clinical development such as in phase 1 studies, PK data is analysed using non-compartmental analysis (NCA) techniques to quantify drug exposures. However, NCA requires sufficient temporal resolution in the PK samples for the underlying numerical approximations to work effectively, hence introducing potential patient burden as well as costs. While there have been work proposing the use of DL algorithms to reduce the number of PK samples required while maintaining accuracy of NCA parameters (e.g., see (Liu, et al. 2024)), there would be additional potential benefits if PK modelling could be done in an automated way. The key reasons for population PK modelling typically performed later in clinical development (e.g., at phase 2 and later) include the human resource burden placed on the limited pharmacometrics teams, as well as the limited ability to model PK data from a few number of subjects and/or PK samples. An appropriately structured PK-NODE *foundation model*, which continuously learns from a growing institutional dataset, could provide a path for PK modelling early in the drug development process, as outlined below.

Firstly, given the expressiveness of NODE formalism as illustrated in Fig. 1, one could develop a single model that covers the dynamics across the different classes of commonly used PK models (e.g., one- and two- and three-compartments) as well as beyond. Thus, such a PK-NODE foundation model removes the need for model selection, but instead an encoder could in essence continuously "steer" the vector field of the NODE model in directions that are needed in order to describe the data. Secondly, through an extensive pre-training on simulations from existing PK model libraries as well as any existing PK data, a PK-NODE *foundation model* could be generated to become an expert PK modeller; moreover, its modelling capabilities could improve over time through the process of *active learning*. Just as a human PK modelling expert with decades of experience modelling across different drug modalities may draw upon her or his wealth of prior professional experience to become significantly more effective compared to a novice at creating models from a limited amount of data, so a foundation model could recognize data patterns and via the intelligence of its encoder, be able to model the PK data from a limited number of patients and/or sparse samples. Such an encoder could leverage not only the temporal PK data, but also any known or estimated physicochemical and/or ADME properties as a *prior* in its modelling process. By significantly reducing the human resource requirements, increasing the throughput as well as data-efficiency of PK and other PMx modelling processes, there is a significant opportunity to broaden the reach of PMx and deepen its impact in drug discovery & development pipeline.

4.2 Improve Predictivity from Limited and "Noisy" Data

Since the inception of PMx as a discipline, the community has evaluated PK and PK/PD models both qualitatively, through the use of a number of diagnostics plots, as well as quantitatively through objective function values. While such techniques have enabled the selection and modification of models (often from an existing library), the evaluation of model predictivity at the individual patient level is often not carried out. However, for advanced applications of PMx modelling, such as in precision medicine or improving decision making in drug development, the ability of the model to predict dynamics at future times is certainly an important characteristic, for which performance measures need to be evaluated. In (Lu, Bender, et al. 2021), it has been shown that from early platelet count data, NODE can enhance predictivity of future platelet counts at the individual patient level. While such enhanced predictivity may not have direct implications for typical PMx applications such as evaluating dosing regimens, this improvement may support improved individual patient dosing in precision medicine applications. In the context of oncology, Tumour Dynamic Neural-ODE (TDNODE) has shown to address the issue of bias in model prediction that has hampered existing tumour dynamics models which have been proposed in PMx community (Turner, et al. 2026). Of note, the identified bias is the most severe when attempting to extrapolate from early in the clinical trial – when predictive modelling to aid decision making would be the most impactful. Therefore, if models such as TDNODE could address the issue of bias and improve the ability to predict future tumour dynamics, this would also likely enhance the operating characteristics of decision-making frameworks that leverage these improved models.

There are also modelling scenarios whereby data has significant inter-individual and temporal variability (i.e., highly "noisy") – for instance, modelling treatment-induced changes in cytokine values. The existing PMx models (such as indirect-response models, etc.) are likely not sufficient for describing them. These are clear opportunities for discovering new PK/PD models that could explain the complexities of such data. Such applications of pushing the boundaries of existing models would fall under the level of "Machine as a Partner" as described in (Terranova, et al. 2024).

In the future, with the evolving landscape of drug discovery and development, PMx models may need to compete with a broader set of computational techniques that have been developed to help enable make better decisions; hence, enhancing predictivity under limited data while addressing bias in predictions would likely take on an increased importance.

4.3 Using High-Content Data

Historically, pharmacodynamic responses are typically scalar quantities that reflect specific molecular (e.g., targeted proteins or metabolites) or cellular abundances (e.g., platelet counts). However, with the technological developments over the past decades, the advent of high-content modalities – ranging from multi-omics to imaging platforms – has enabled the capture of high-dimensional pharmacological responses that reveal systems-level perturbation signatures. Such datasets provide rich potential insights for drug discovery & development applications and new opportunities for the future of PMx.

High-content data should not be treated simply as concatenation of scalar quantities; in fact, there is expected to be certain correlations within components of such data, as well as noise that need to be accounted for. To use such data, one should leverage the concept of learned *vector embeddings*, which represent the lower dimensional manifold within which the high-content data evolves (see Panel B of Fig. 7). Under this view, the PD would be represented as a point in an abstract space that evolves under the driving influence of PK – from which PK/PD concepts would be applied.

6. Implementation Considerations

Transitioning from traditional PMx software (e.g., NONMEM, R, Monolix) to Neural-ODE frameworks requires adapting to entirely different software ecosystems: e.g., based on Julia or Python programming languages.

6.1 Software and Frameworks

In the PyTorch ecosystem, `torchdiffeq` is the standard library for solving Neural-ODEs, while `torchsde` is utilized for Neural Stochastic Differential Equations (SDEs) when modelling process noise. These libraries are highly optimized for deep learning, leveraging PyTorch's robust automatic differentiation (autodiff) and GPU acceleration.

An important note in integrating `torchdiffeq` into a standard PMx workflow requires addressing a conceptual gap: the former is designed to solve systems of the form $dy(t)/dt = f_\theta\big(y(t)\big)$, hence there is no notion of (individual-level) model parameters, p. Instead, only global weights θ are represents the global weights of the neural network. Hence, a workaround proposed in (Laurie and Lu 2023) is to introduce an augmented state, $\tilde{y} = [y, p]^T$ and enforce p being a time-independent parameter by appending the differential equation, $dp/dt = 0$.

In Julia's SciML ecosystem `DiffEqFlux.jl`, by design the implementation is well aligned with traditional PMx tools. Because Julia's differential equation solvers (`DifferentialEquations.jl`) natively support parameterized ODEs of the form as shown in eqn (2), integrating parameters p into the differential equation model in a programmatically seamless manner.

There is a point of note in the training of NODEs: given the propensity of dynamical systems to exponentially blow up in time and/or the underlying numerical ODE solvers experiencing challenges in selecting time step sizes that meeting the convergence criteria, some thought need to be given to the choice of initializing the initial set of weights and biases as well as the learning rate. While this is not a wholly solved problem and robust approaches have been demonstrated across classes of problems, it is not thought to be an inherent limitation of NODEs but is expected to be solved with further research.

6.2 Computational Cost and Scalability

For small NODEs (modest state/parameter dimensions), GPUs typically offer little benefit over CPU, because adaptive-step solvers are inherently sequential in time and the matrix multiplications performed at each time step are too small to be efficiently accelerated by GPU. Hence, unless specialized systems and simulations are considered, such as solving extremely large systems and generating ensembles of simulations, CPUs may work just as well as GPUs.

7. Challenges and Limitations

While the NODE formulation has built into it the concept of dynamical systems and hence the well-established ways to understand and use them, understanding the vector field that has been learned is non-trivial. To advance human scientific understanding from the learned models, such

as figuring out why TNODE models may better extrapolate tumour dynamic trajectories (Turner, et al. 2026), further analyses would be warranted to look beneath the neural network weights. Hence, methods such as symbolic regression, or extensions thereof, need to be applied.

Another aspect of NODE formulation as discussed in this tutorial that may need to be extended for certain PMx applications is the notion of population modelling. In particular, while the existing formulation learns a single dynamical system from the population data and with each patient being represented with individual parameter vectors $p \in \mathbb{R}^m$, there is no notion being introduced that they lie within certain pre-specified statistical distributions. While placing a constraint on the distribution for p could be done, given that they may have physical meanings that are yet to be discovered, it goes against having an *a priori* distribution placed on them. Whether pre-specified statistical distributions are needed or whether they should also be learned, remains a scientific topic for further investigation.

Conclusions

By its formulation, NODE sits at the interface between classical dynamical modelling as employed by the field of PMx, and DL. By extending the language of dynamical models from algebraic expressions to neural networks (see Panel A of Fig. 7) as well as generalize its concepts to advanced high-content data (see Panel B of Fig. 7), NODEs offer the potential to extend the reach of PMx to new domains. As AI and recent technological forces are jointly reshaping the landscape of drug discovery & development, NODEs offer a path forward to meet the future expectations for the discipline of PMx.

Given the fact that NODE is both a DL and a dynamical systems model, care needs to be taken in making the most out of the limited training data through data augmentation, as well as posing the problem using the appropriate dynamical systems concepts. For NODEs to make the most impact, one needs to look beyond using it simply as a "drop-in" for an existing PMx model and workflow. In particular, if one were to seek gains in modelling efficiency, one needs to consider using NODEs as a foundational model, within a *zero-shot* or *few-shot* learning workflow. In terms of applications, one should carefully consider the *why* of using NODE: is it to make the workflow more efficient, or overcome a key limitation of existing PMx model, or work with high-content data in a way that would not have been feasible with PMx modelling? The reason for using NODE may also determine how to architect the appropriate workflow.

As we have entered the age whereby agentic AI is being broadly used to perform and orchestrate computational workflows in biomedical and drug discovery application, it will become an expectation that pharmacometricians augment their intelligence using AI agents (see Panel C, Fig. 7). As AI agents become increasingly proficient at building models in PMx and QSP (Lu and Desikan, Quantitative Systems Pharmacology Modeling Amid the Rise of Agentic AI 2026), the technical barriers for using NODEs will also be lowered. However, understanding the key

concepts and ensuring best practices are adopted will nevertheless be relevant for the human modellers in order to ensure the right conclusions are drawn from the modelling results. As NODEs become part of the common modelling skillset for humans and AI agents alike, we expect further opportunities for impact will open up for the PMX community.

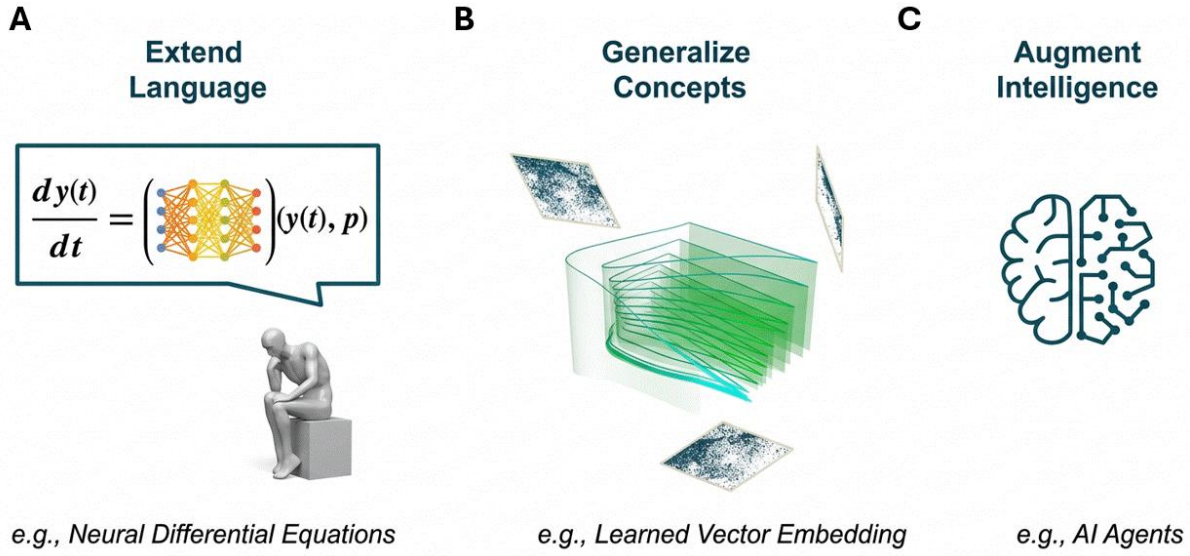


Figure 7. Three ways to leverage AI to advance pharmacometrics.

References

Bazgir, Omid, Zichen Wang, Ji Won Park, Marc Hafner, and James Lu. 2023. "Integration of graph neural network and neural-odes for tumor dynamic prediction." *preprint arXiv:2310.00926.*

Chen, Ricky TQ, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. 2018. "Neural ordinary differential equations." *Advances in neural information processing systems.*

Hossain, Intekhab, Viola Fanfani, John Quackenbush, and Rebekka Burkholz. 2024. "Biologically informed NeuralODEs for genome-wide regulatory dynamics." *Genome Biology.*

Laurie, Mark, and James Lu. 2023. "Explainable deep learning for tumor dynamic modeling and overall survival prediction using Neural-ODE." *NPJ Systems Biology and Applications.*

Liu, Gengbo, Logan Brooks, John Canty, Dan Lu, Jin Y. Jin, and James Lu. 2024. "Deep-NCA: A deep learning methodology for performing noncompartmental analysis of pharmacokinetic data." *CPT: Pharmacometrics & Systems Pharmacology.*

Lu, James, and Rajat Desikan. 2026. "Quantitative Systems Pharmacology Modeling Amid the Rise of Agentic AI." *CPT: Pharmacometrics & Systems Pharmacology.*

Lu, James, Brendan Bender, Jin Y Jin, and Yuanfang Guan. 2021. "Deep learning prediction of patient response time course from early data via neural-pharmacokinetic/pharmacodynamic modelling." *Nature machine intelligence.*

Lu, James, Kaiwen Deng, Xinyuan Zhang, Gengbo Liu, and Yuanfang Guan. 2021. “Neural-ODE for pharmacokinetics modeling and its advantage to alternative machine learning models in predicting new dosing regimens.” *iScience.*

Pakravan, Samira, Nikolaos Evangelou, Maxime Usdin, Logan Brooks, and James Lu. 2024. “From Noise to Signal: Unveiling Treatment Effects from Digital Health Data through Pharmacology-Informed Neural-SDE.” *arXiv preprint arXiv:2403.03274.* https://arxiv.org/abs/2403.03274.

Rossi, Emanuele, Bertrand Charpentier, Francesco Di Giovanni, Fabrizio Frasca, Stephan Günnemann, and Michael M. Bronstein. 2024. “ Edge directionality improves learning on heterophilic graphs.” *Learning on graphs conference.* PMLR.

Shahin, Mohamed H, Prashant Desai, Nadia Terranova, Yuanfang Guan, Tomáš Helikar, Sebastian Lobentanzer, Qi Liu, James Lu, Subha Madhavan, and Gary Mo. 2025. “AI-Driven Applications in Clinical Pharmacology and Translational Science: Insights From the ASCPT 2024 AI Preconference.” *Clinical and Translational Science.*

Simeoni, Monica, Paolo Magni, Cristiano Cammia, Giuseppe De Nicolao, Valter Croci, Enrico Pesenti, Massimiliano Germani, Italo Poggesi, and Maurizio Rocchetti. 2004. “Predictive pharmacokinetic-pharmacodynamic modeling of tumor growth kinetics in xenograft models after administration of anticancer agents.” *Cancer research* 1094-1101.

Terranova, N., D. Renard, M. H. Shahin, S. Menon, Y. Cao, C. E. Hop, ..., and J. Lu. 2024. “Artificial intelligence for quantitative modeling in drug discovery and development: an innovation and quality consortium perspective on use cases and best practices.” *Clinical Pharmacology & Therapeutics.*

Turner, David C, Mark Laurie, Shuhui Li, Huy X Ngo, Chunze Li, Jin Y Jin, James Lu, and Logan Brooks. 2026. “Follow-Up Bias in Tumor Dynamic Modeling: A Comparison of Classical and Neural-ODE Approaches.” *CPT: Pharmacometrics & Systems Pharmacology* e70239.